Embedded Data Structure
(Lecture Note 24)

Searching Algorithms (7)

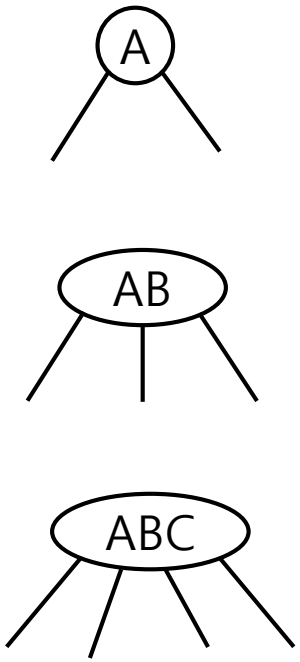
2025. 6. 10

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

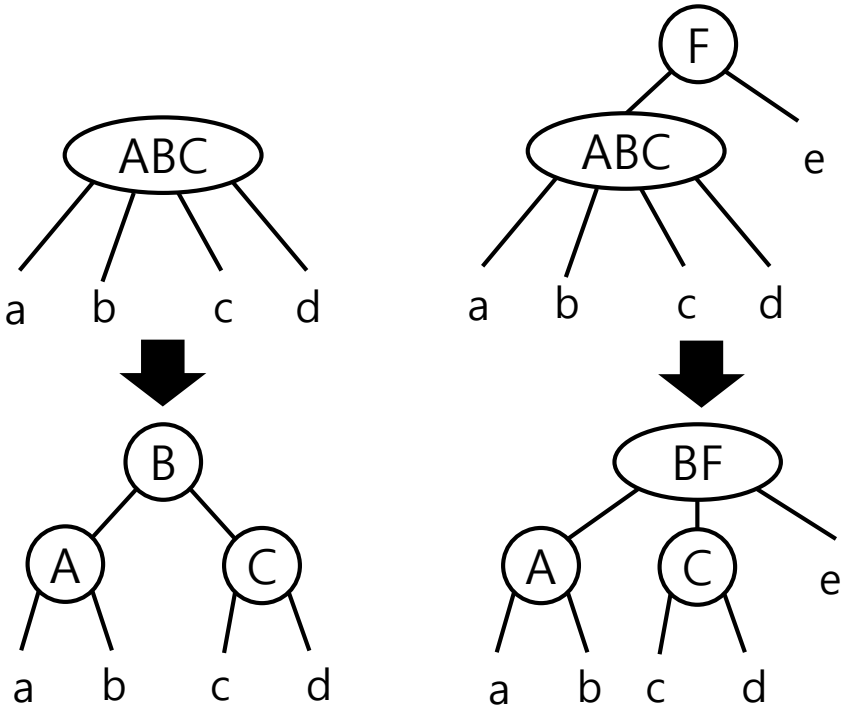
2-3-4 Tree (1/7)

- **Three types of nodes**
  - **Estimate the mid. point using the interpolation**



Three types of nodes

Split for balancing a tree :

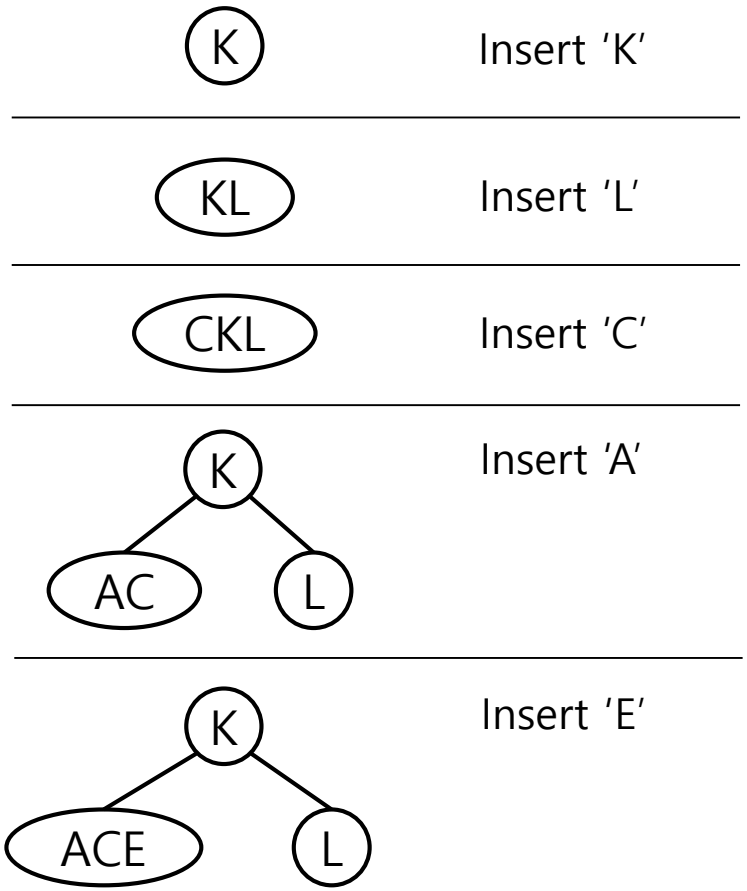


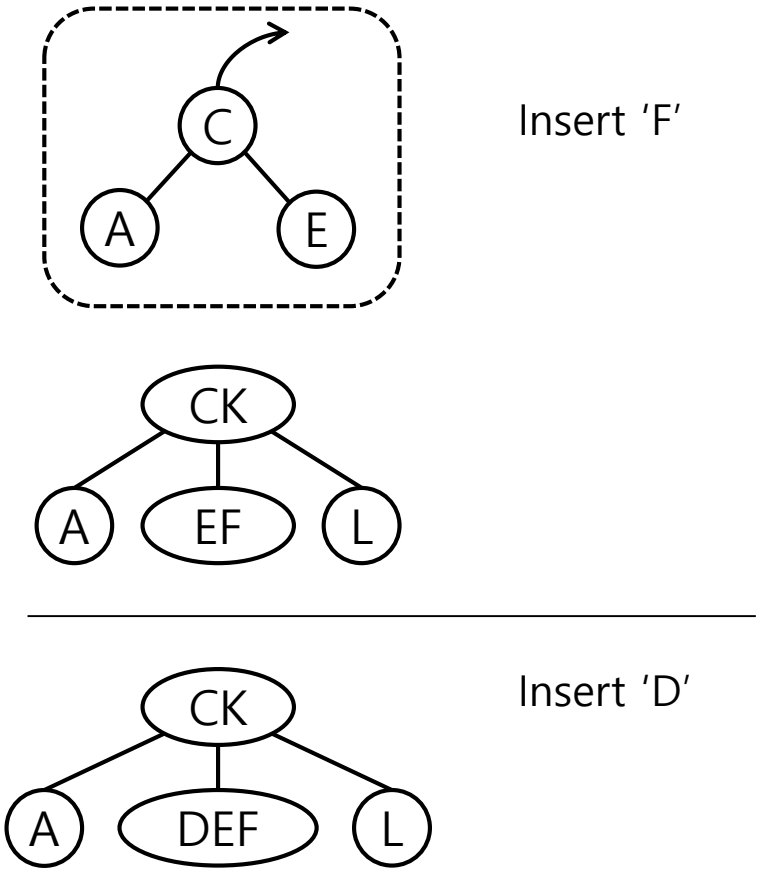
Root (4-node)

General case

2-3-4 Tree (2/7)

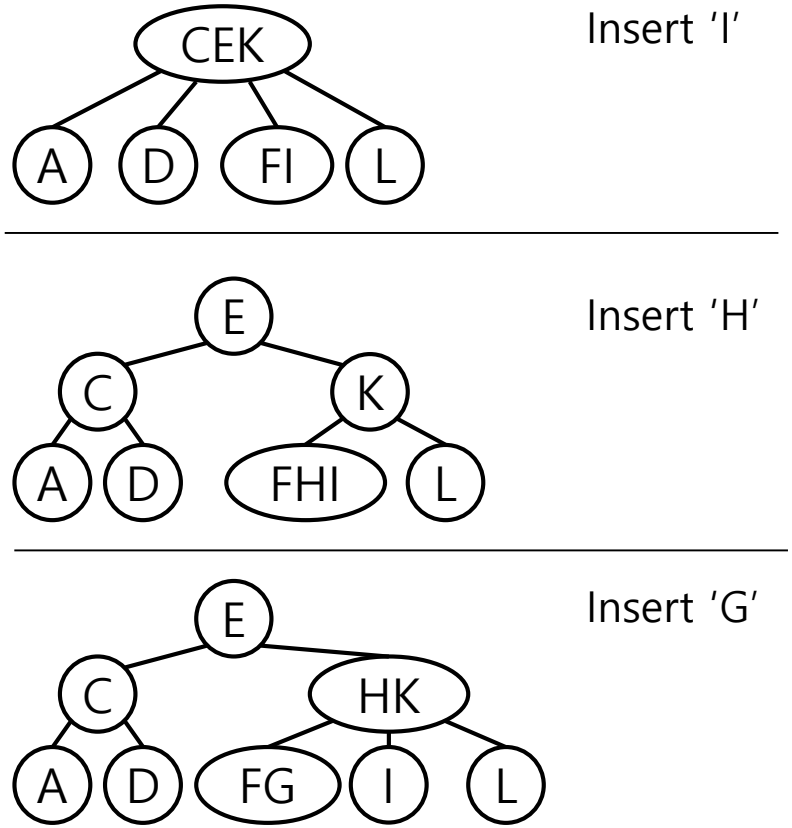
- **Insertion procedure**
  - **Start from the external node**





2-3-4 Tree (3/7)

- **Insertion procedure – cont'd**
  - **Remember the split strategy**



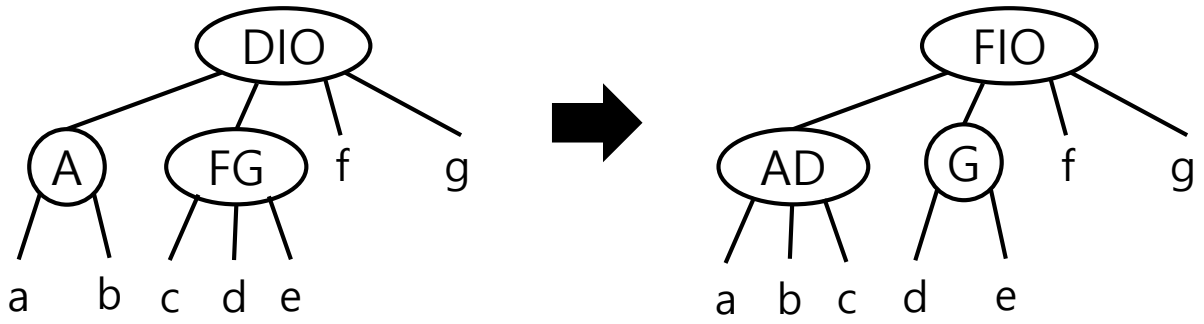
Overall procedure (insert) :

1. while(cur.node != ext.node)
   1.1 if cur.node is 4-node, split
   1.2 if key < 1st val, go to 1st child
   1.3 if key < 2nd val, go to 2nd child
   1.4 if key < 3rd val, go to 3rd child
   1.5 if key >= 3rd val, go to 4th child
2. If ext.node is 4-node, split
3. Merge key into ext.node

2-3-4 Tree (4/7)

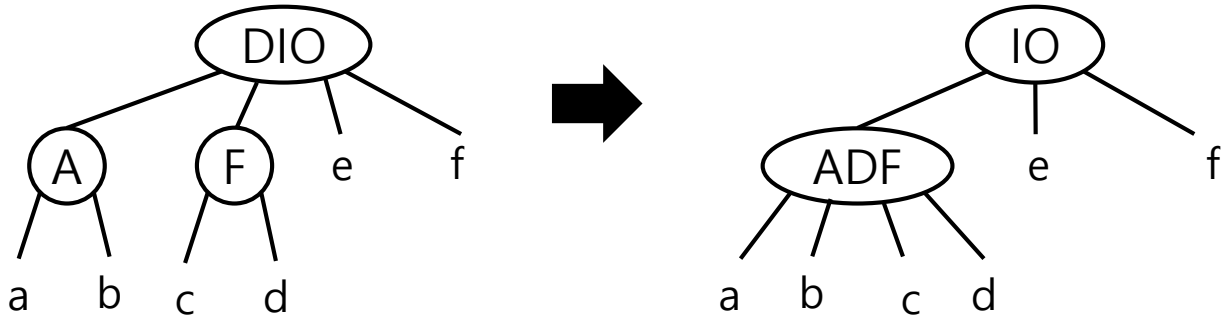
- **Deletion procedure (external node)**
  - **Different approach btw 2-node and 3-/4-node**

Delete 'A' :



Convert A node to 3-node by borrowing val.

Delete 'A' :

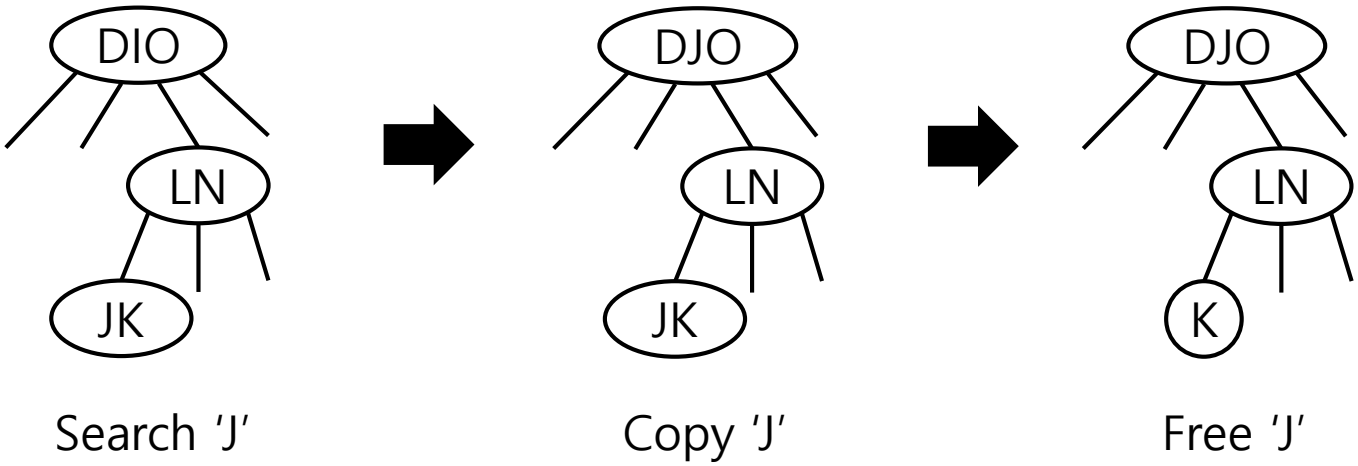


Convert A node to 4-node by merging val.

▪ Deletion procedure (internal node)

- **Similar scheme with deletion of binary tree search**

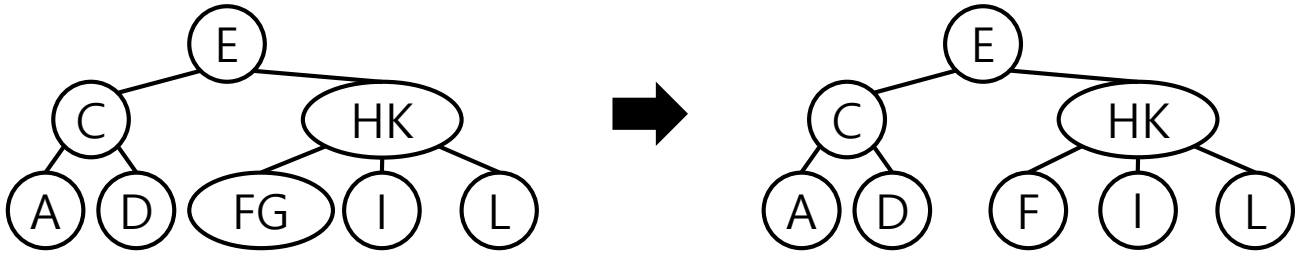
Delete 'I' :



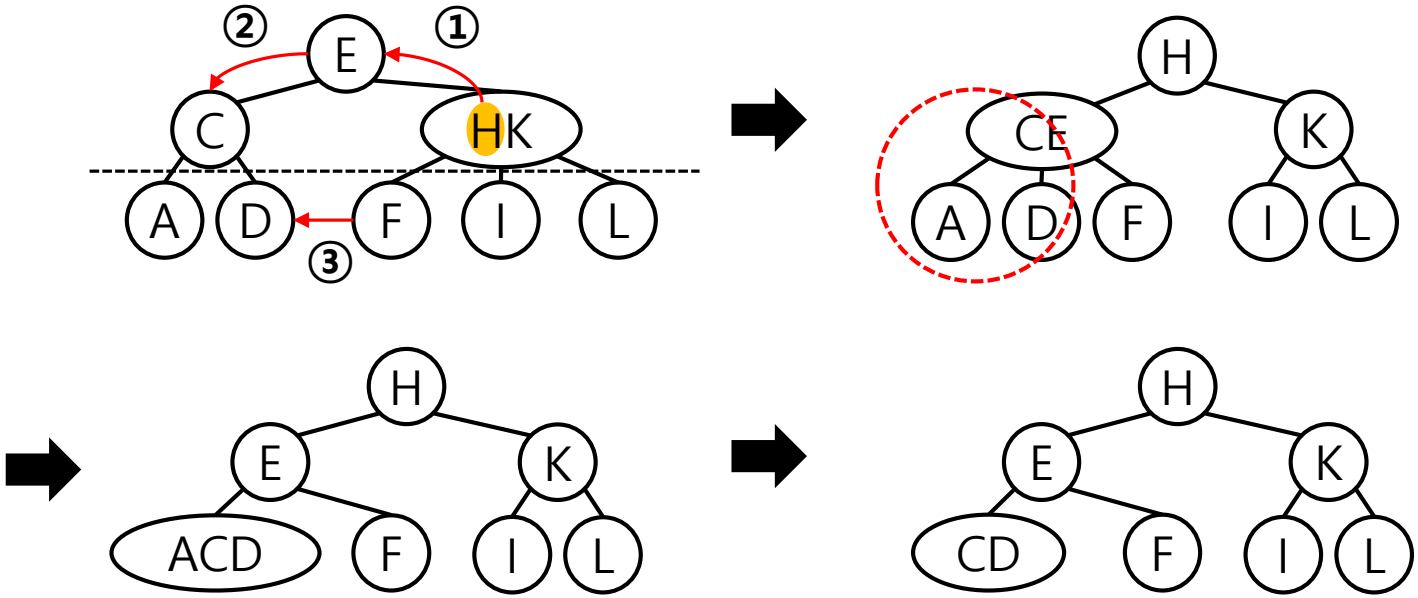
- Search the nearest node compared to a given key
- Delete a given key and copy that node to the current position
- Free the selected node

2-3-4 Tree (6/7)

- **Deletion procedure (total example)**



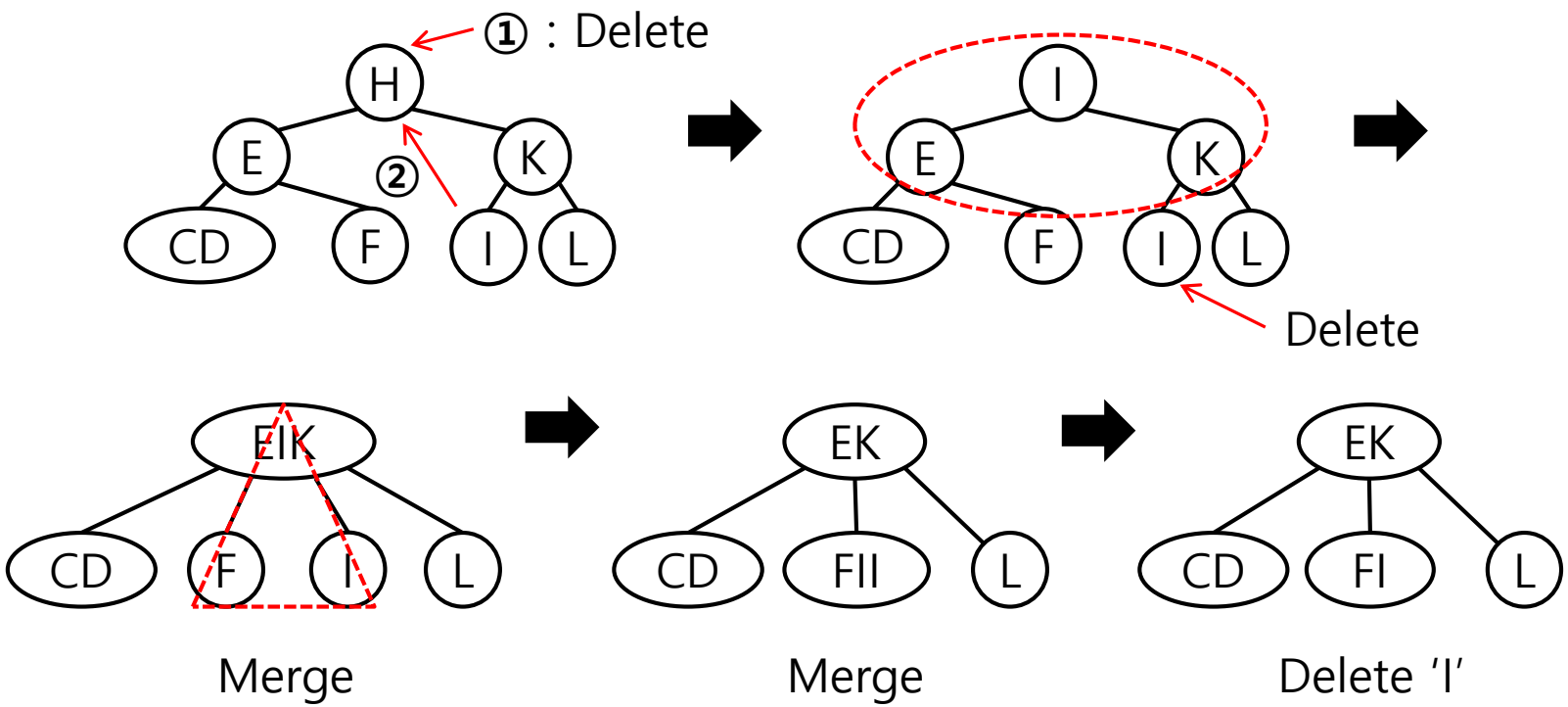
[Delete 'G']



[Delete 'A']

2-3-4 Tree (7/7)

- **Deletion procedure (total example) – cont'd**



R-B Tree : Concept (1/11)

- **Basic rule : left child < parent < right child**
  - **We use colors (red and black) for representing "split" in 2-3-4**

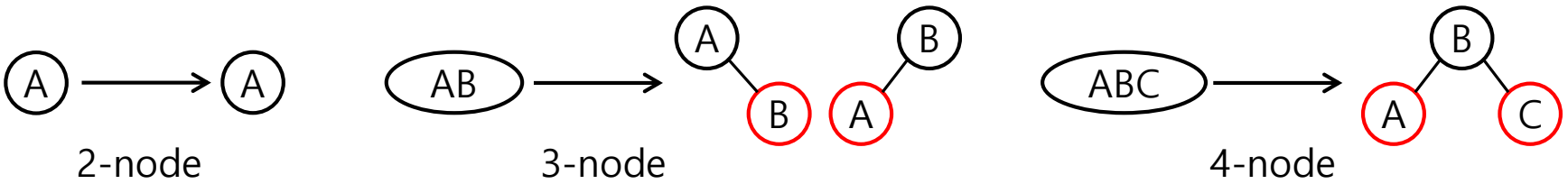
Properties of R-B tree :

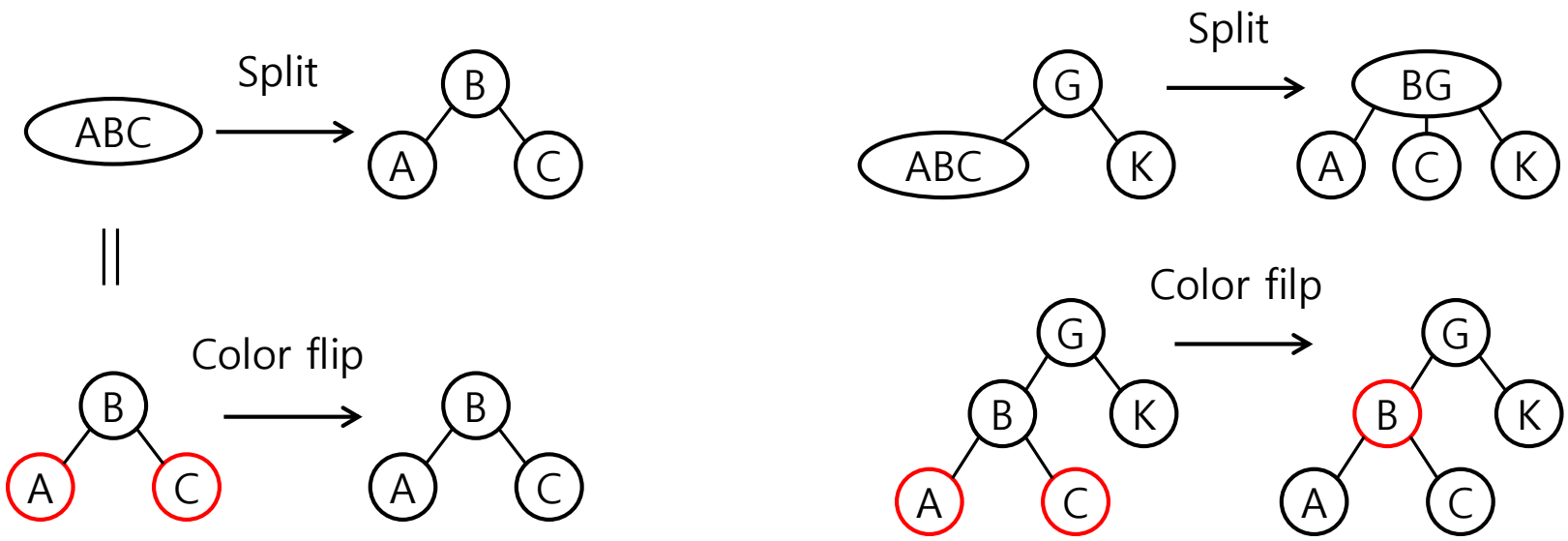
1. The number of black nodes are same to go to the external nodes
2. When node is inserted → red
3. Two consecutive reds are not allowed (rotation is required)
4. Two children are red, then **color flip** is conducted (parent ↔ children)
5. Root cannot be red (if red, then change to black)
6. Red nodes do not have child or both needs to be black
7. Black nodes do not have child or one red / two reds / two blacks

➡ **Remember rules for R-B tree above !**

Representation

- **Key point : split in 2-3-4 tree ➔ color flip in R-B tree**

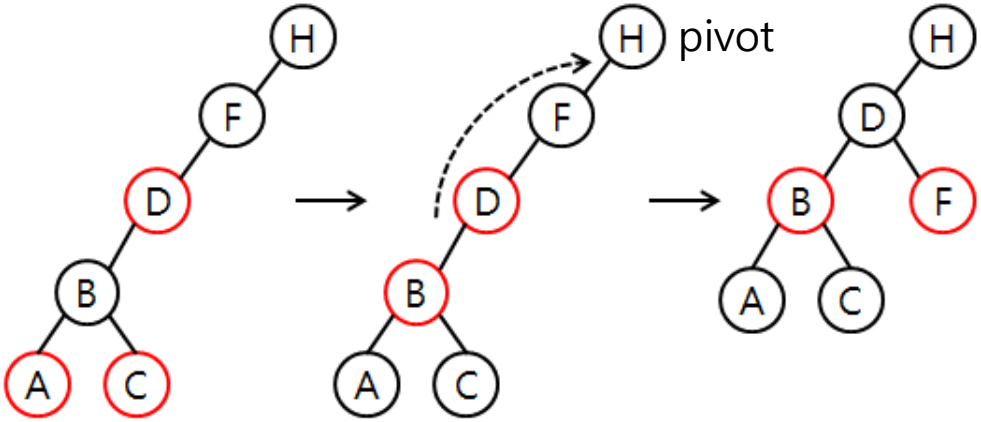




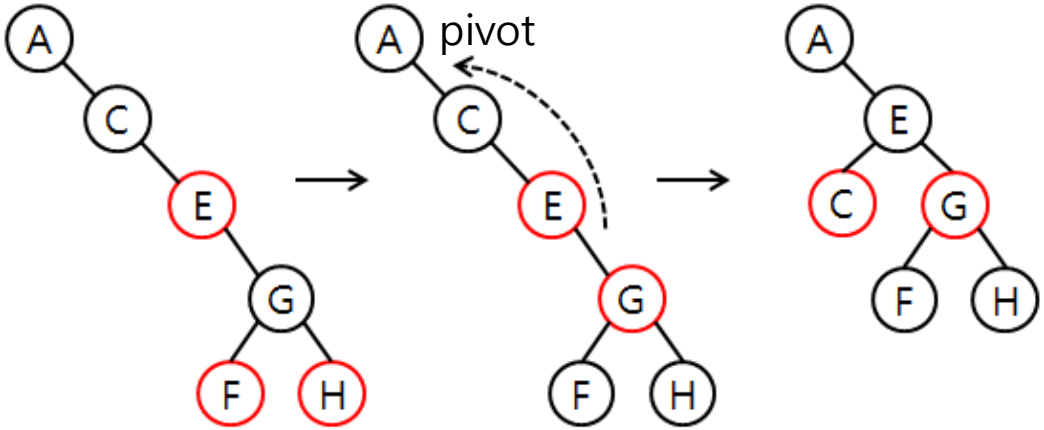
R-B Tree : Concept (3/11)

- **Rotation : making balanced of red-red nodes**
  - **There are four types of rotations (LL, RR, LR, RL)**

LL rotation :



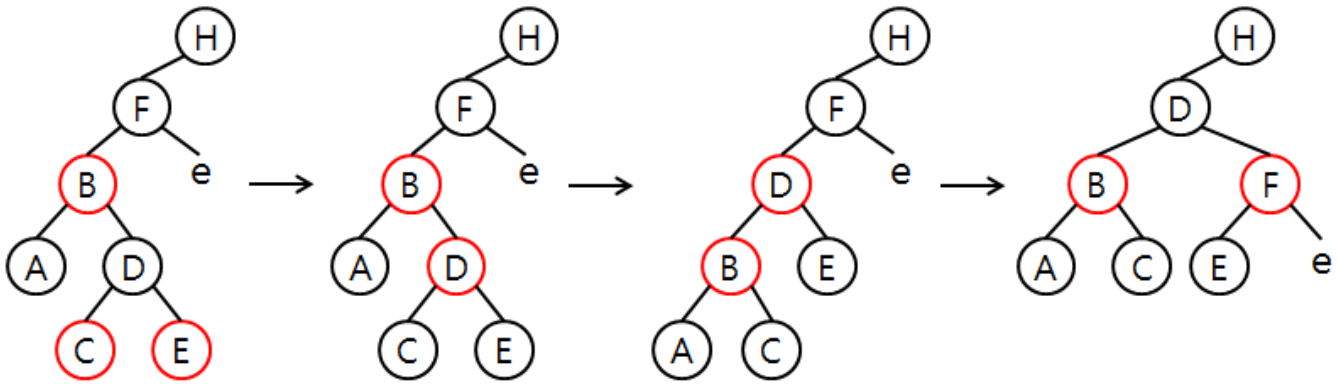
RR rotation :



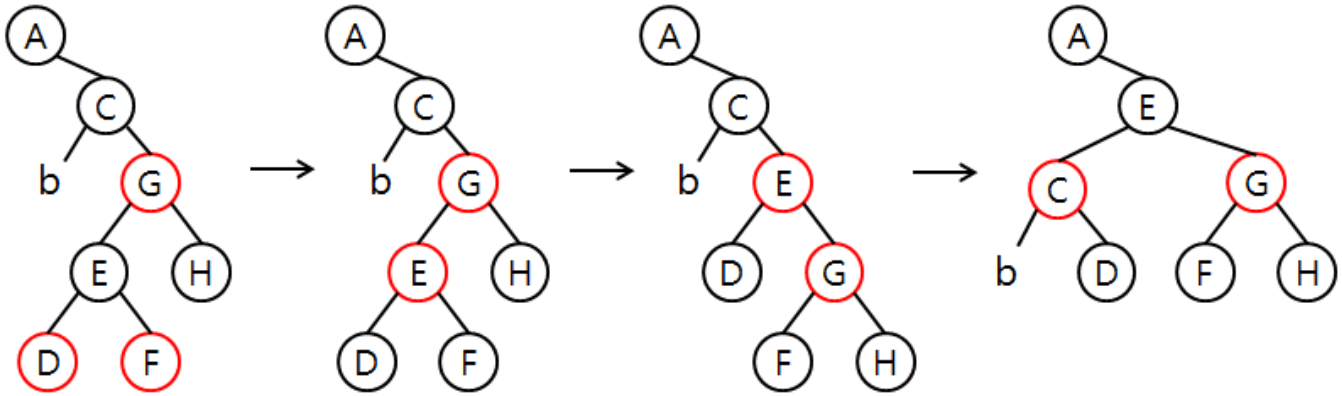
R-B Tree : Concept (4/11)

- **Rotation : making balanced of red-red nodes – cont'd**
  - **More complex rotations : LR and RL**

LR rotation :



RL rotation :

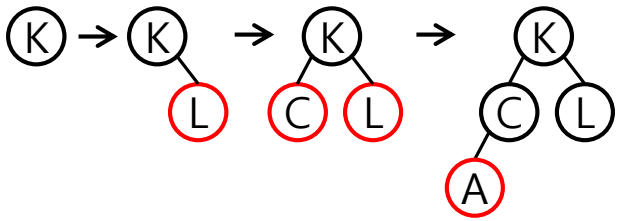


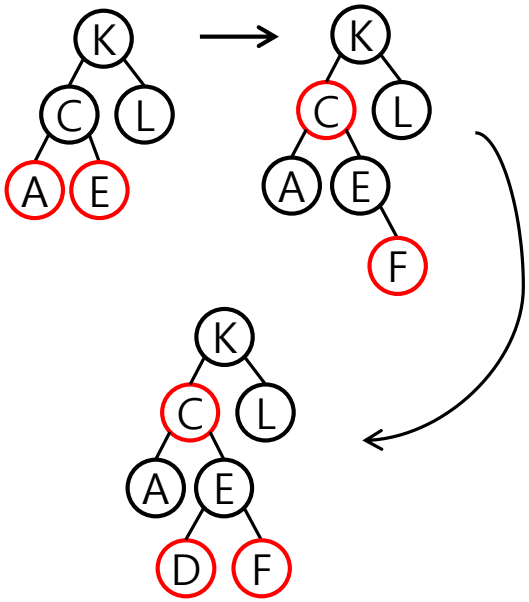
Node insertion

- **If needed, rotations are conducted**

Insertion procedure :

1. If cur node is not external,
   1. If two children are red, then color flip
   2. After flipping, if needed, rotation is conducted
   3. If key < cur node, go left
   4. If key >= cur node, go right
2. Insert a red node (if needed, rotation is conducted)

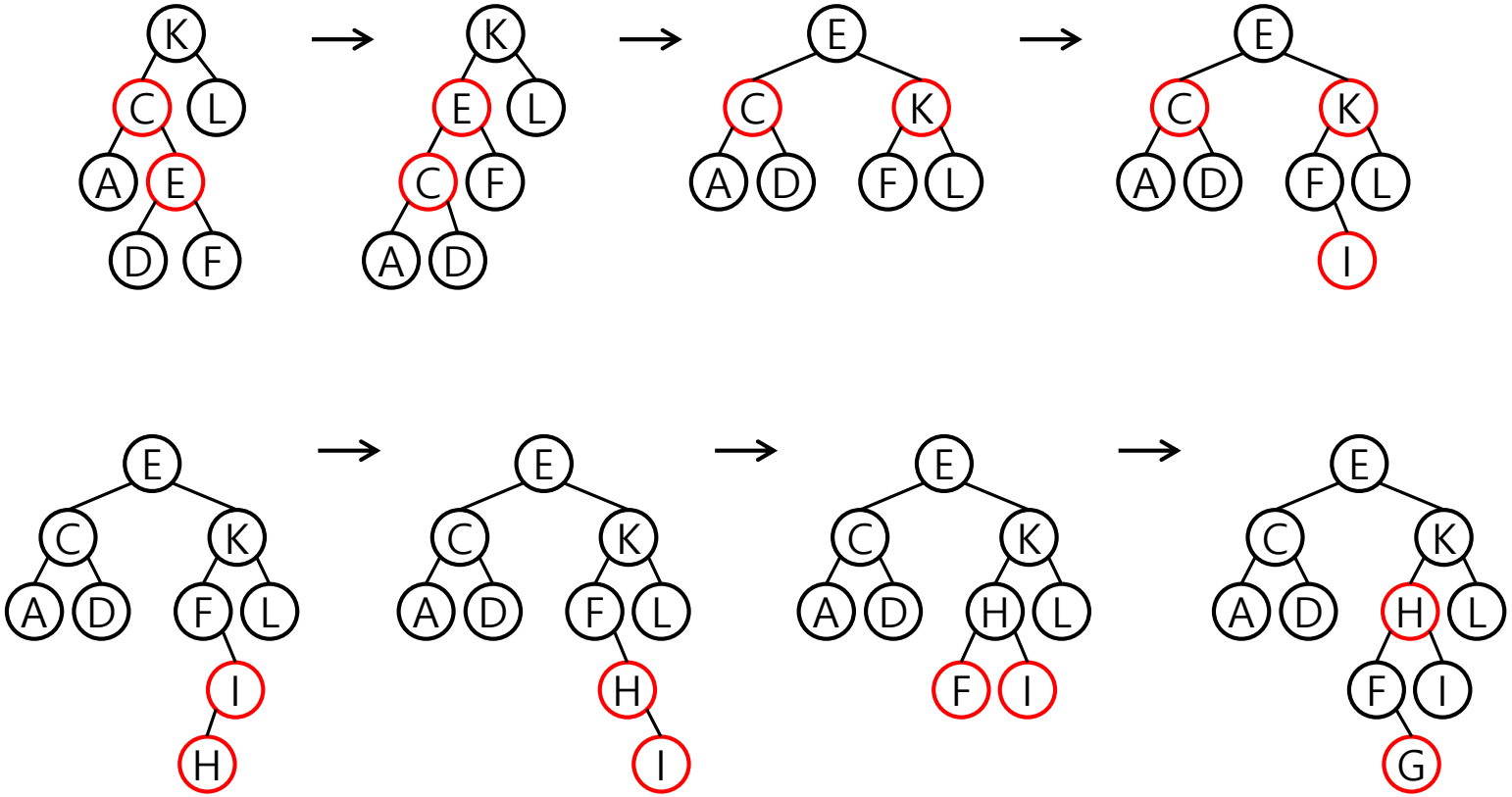




Insert : K L C A E F D I H G

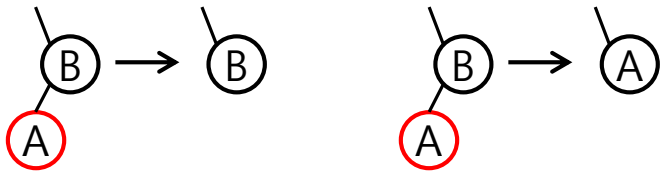
R-B Tree : Concept (6/11)

- **Insertion process**
  - **Focus on color clip and rotation**



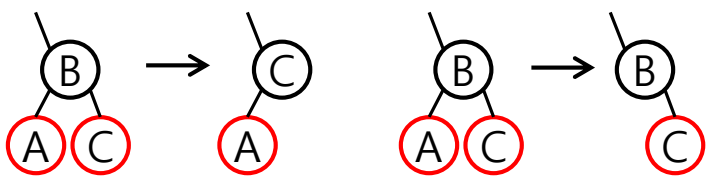
▪ Node deletion

- **Deletion for 3-node and 4-node / 2-node**



Delete 'A' Delete 'B'

[3-node]



Delete 'B' Delete 'A'

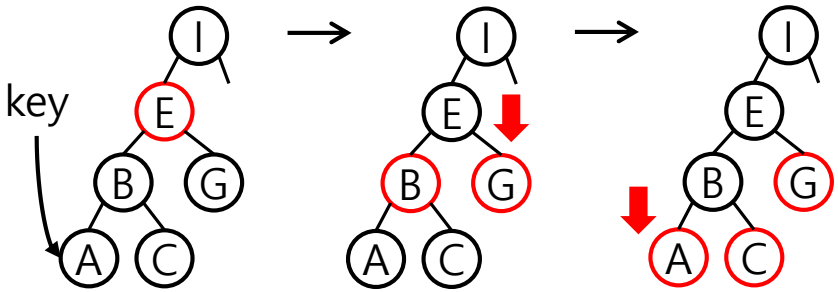
[4-node]

Case of 2-node :

In 2-3-4 tree, borrowing or merging

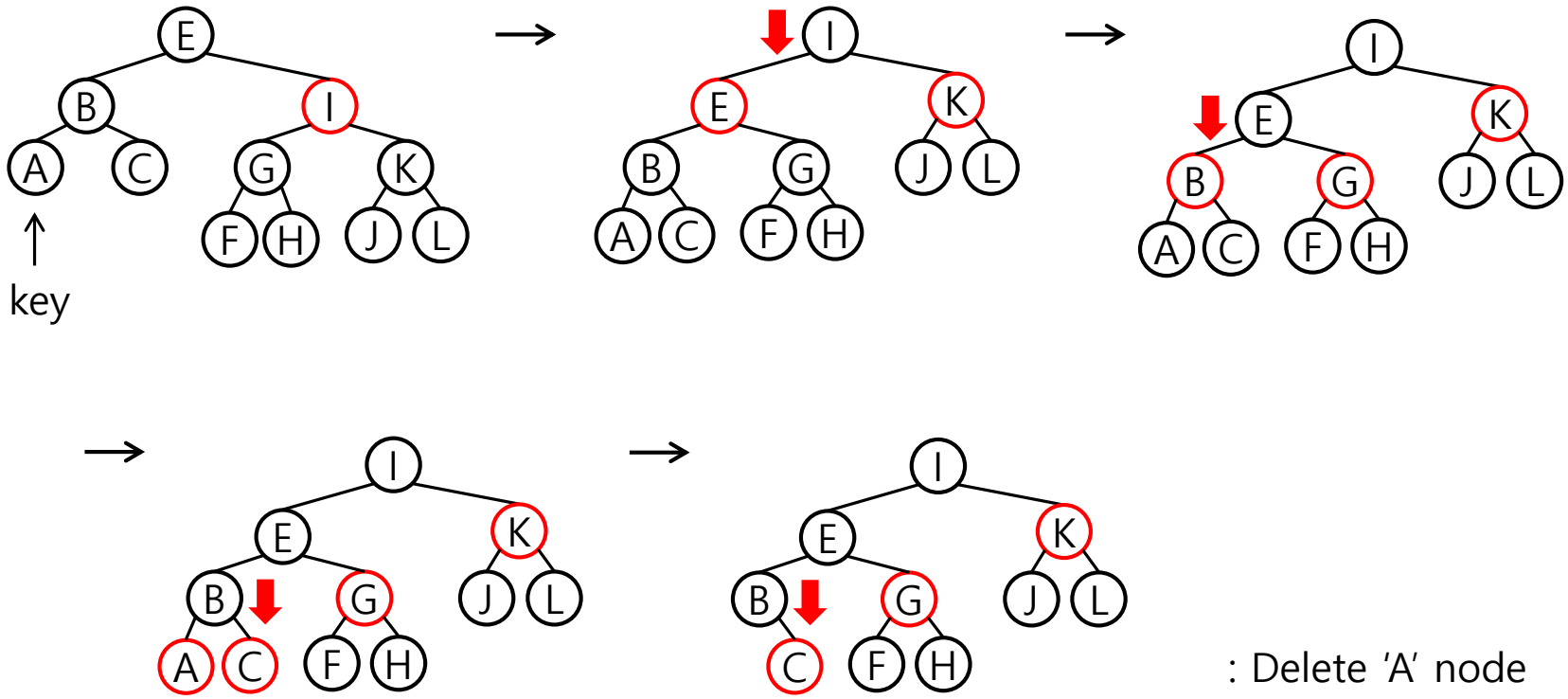
In R-B tree, **color de-flip**

➔ Find the red node (**seed**) at first !

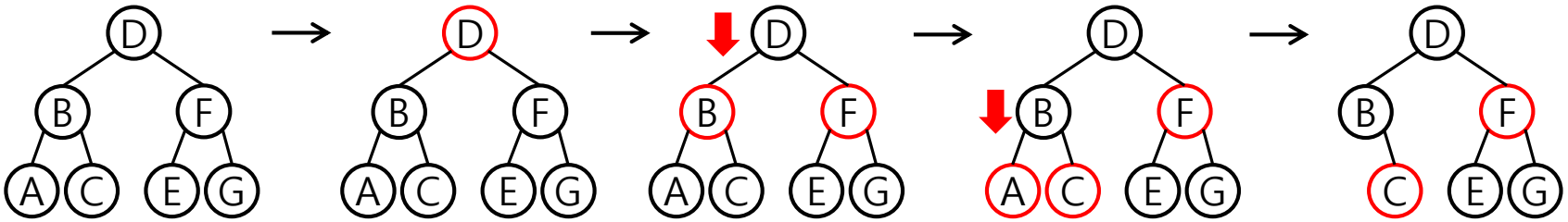


: De-flip until meeting the target key

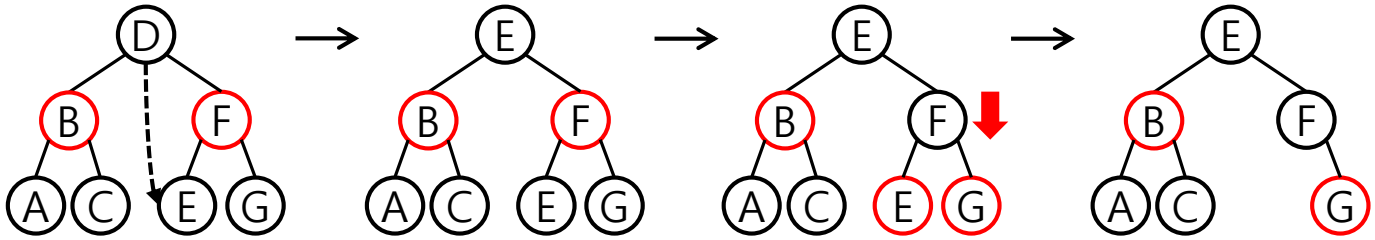
- **Node deletion – cont'd**
  - **Find red seed in the family**



- **Node deletion – cont'd**
  - **No red seed / internal node**



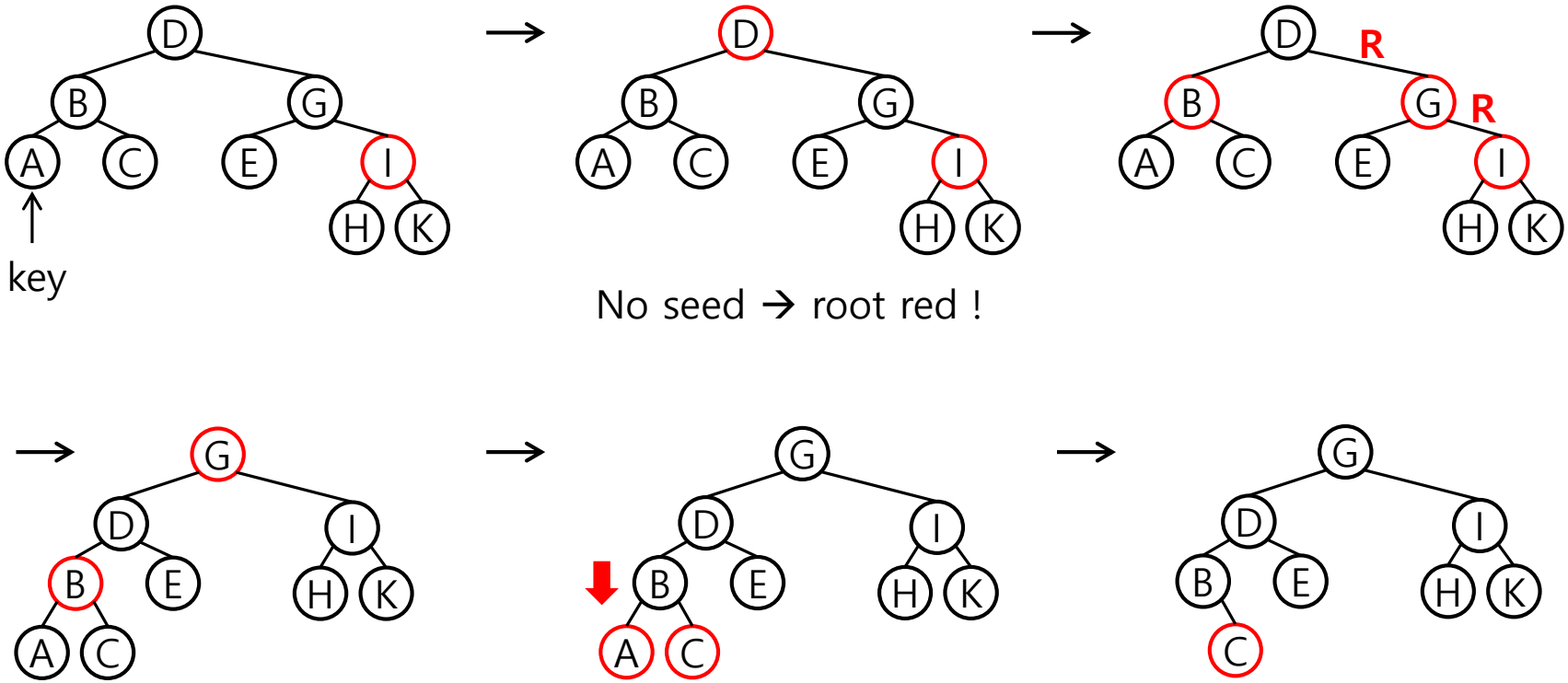
Delete 'A' node when there is no red seed (➔ **making a root red !**)



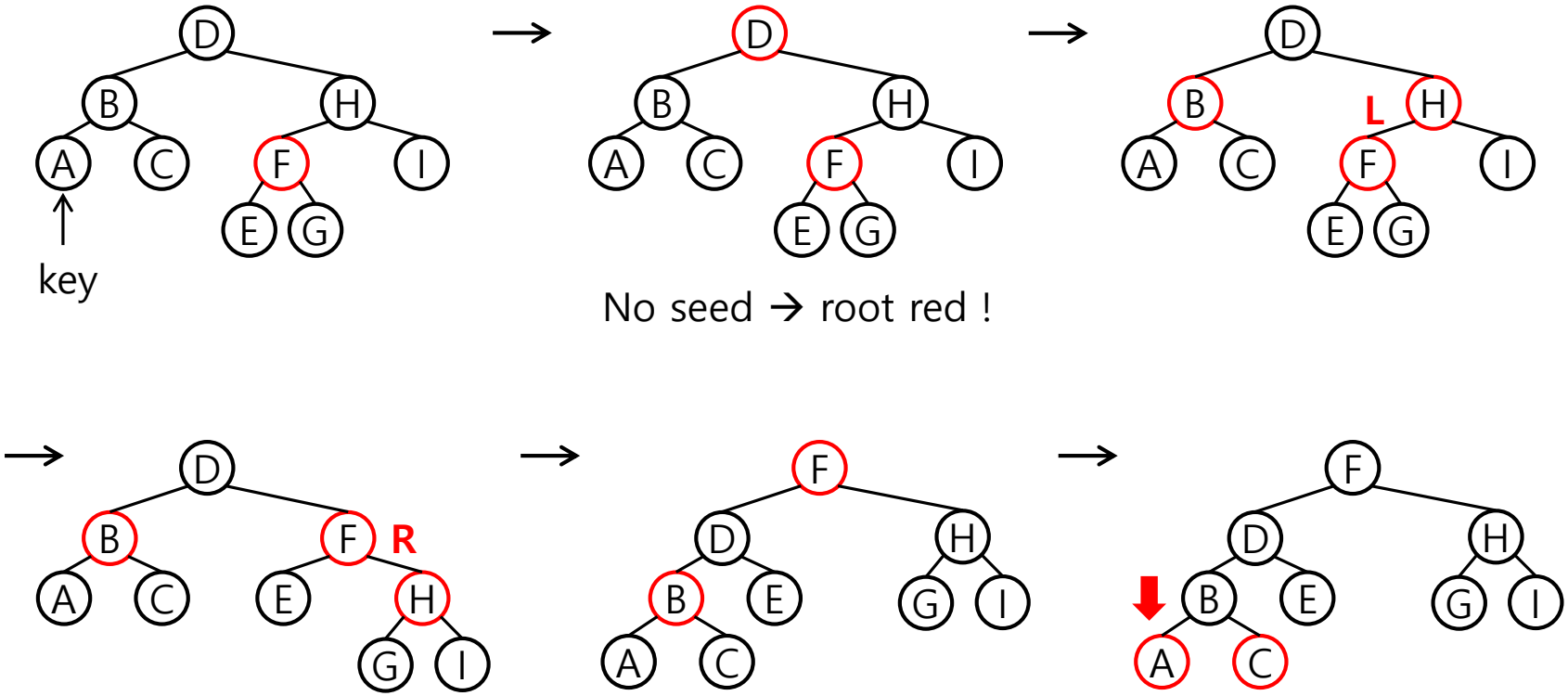
Delete 'D' node (internal node) by replacing the nearest neighbor

Node deletion – cont'd

- **Deletion with rotation (for example, RR)**



- **Node deletion – cont'd**
  - **Deletion with rotation (for example, RL)**



Summary

- **More than two links …**
  - **Concept of the 2-3-4 tree**
  - **Concept of the red-black tree**